D001

Architecture Foundation and Core Contract Layer

D001 freezes the engine vocabulary, ownership model, and dependency rules that later milestones will inherit. It is intentionally architecture-only: no runtime mechanics, no rendering detail, no gameplay logic.

Field	Summary
Purpose	Define the core architectural spine and naming rules.
Scope	Shared contracts, module ownership, configuration policy, diagnostics, and freeze criteria.
Outcome	A stable contract layer that later milestones can bind to without redesign.
Status	Freeze-ready baseline for the rest of the series.

This document is written for review. It keeps implementation detail out of view so the team can approve the foundation before subsystems begin to accumulate local assumptions.

1. Executive Scope

D001 owns the project spine: identity, shared contracts, composition conventions, lifecycle language, and diagnostics. It does not own rendering, physics, content, or platform adapters.

Boundary clarity matters because an under-specified foundation causes drift, while an over-specified one starts competing with later milestones.

D001 should feel small enough to finish and large enough to matter.

2. Design Principles

The foundation is conservative by design. It favors explicit dependencies, small surfaces, and single ownership over clever abstraction.

If a change weakens one of these principles, the change must prove why the tradeoff is worth it.

Principles are review criteria, not slogans.

3. Logical Architecture

The host sits outside the engine core. The composition root assembles concrete services. The contract layer defines modules, policy, lifecycle, and diagnostics. Adapters stay at the edge.

The diagram shows ownership direction, not the full product surface.

Layer	Role	Rule
Host	Outer application or tool.	Outside the engine.
Composition root	Creates concrete services.	Owns assembly.
Core contracts	Shared language and policy.	Frozen foundation.
Feature modules and adapters	Depend on the contracts.	Extend without rewriting.

The important rule is inward-facing language and outward-facing adapters.

4. Module Boundaries

A module is a responsibility boundary with a named contract, not just a folder. D001 defines the core set: identity, module contract, composition contract, configuration contract, diagnostics contract.

The purpose is to eliminate ambiguity and prevent accidental coupling.

Area	D001 ownership	Later use
Identity and metadata	Version, label, compatibility notes.	Docs and tooling.
Module contract	Readiness and declared dependencies.	Feature registration.
Configuration contract	Layered validation rules.	Runtime settings.
Diagnostics contract	Error and status snapshots.	Logging and troubleshooting.

A later milestone must be able to name the contract it depends on.

5. Dependency Flow

The dependency graph must run one way: host -> composition root -> core contracts -> feature modules and adapters. Information moves through explicit contracts only.

This keeps bootstrap predictable and reviewable.

No module should need to know where the host lives or how it was launched.

6. API Surface

D001 freezes the public vocabulary: EngineDescriptor, ModuleDescriptor, CompositionContext, EngineConfig, EngineResult, EngineStatus, CapabilityDescriptor, and DiagnosticEvent.

These are architectural commitments, not code samples.

Contract	Intent
EngineDescriptor	Identity and capability summary.
ModuleDescriptor	Module readiness and scope.
CompositionContext	Host services and bootstrap inputs.
EngineConfig	Resolved validated configuration.
DiagnosticEvent	Structured warning/error/status.

Every object should be constructible from explicit inputs and safe as a read-only snapshot.

7. Lifecycle Model

The engine needs a state language before it has a runtime. D001 establishes Created, Configured, Validated, Ready, and Frozen so later milestones do not rename them.

Transitions must be guarded and explainable.

State	Meaning
Created	Accepts composition inputs and identity.
Configured	Layers are merged and normalized.
Validated	Failures are surfaced before activation.
Ready	The foundation can be consumed by later milestones.
Frozen	Contract terms are stable and review-approved.

Frozen means the contract is stable enough for downstream dependency.

8. Configuration Policy

Configuration is a merging and validation policy, not a file format. The precedence is defaults, environment, host overrides, then milestone-specific additions.

Invalid values must fail before readiness.

Policy	Rule
Source order	Defaults, environment, host, then milestone overrides.
Validation	Reject incompatible values before ready.
Snapshot rule	Expose immutable resolved configuration.
Override policy	Do not silently replace required fields.

Consumers receive an immutable resolved snapshot, not a live bag of settings.

9. Errors and Diagnostics

Errors and diagnostics share one architecture language: stable codes, severity, recoverability, and context payloads.

A failure must say what failed, how serious it is, and which layer owns it.

Field	Meaning
Error code	Stable identifier for tooling.
Severity	Info, warning, recoverable, fatal.
Recoverability	Continue, retry, or stop.
Context payload	Module, phase, input layer, field.
Correlation	Optional trace identifier.

This is the starting point for observability and tooling.

10. Later Milestones

D001 must remain useful when runtime, scenes, input, rendering, audio, and tooling arrive. Those milestones own behavior; D001 owns the shared vocabulary.

The contract layer protects the roadmap by giving later work a stable reference point.

Area	D001 ownership	Later use
Runtime and scheduling	Lifecycle state, clock, event envelope	D007 and control loops
Scenes and content	Registration and capability rules	Scene orchestration
Input and commands	Command envelope and host adapters	Input routing
Rendering and audio	Adapter contracts and status reporting	Presentation layers
Tooling and automation	Metadata and freeze criteria	Build/test/publish tools

Downstream systems should bind to D001, not reinterpret it.

11. Rationale

The design is intentionally conservative. It avoids feature logic and scheduling detail so the foundation can survive multiple implementation passes.

Early clarity is worth more than early completeness.

The project can add capability later; it cannot easily repair a confused foundation.

12. Risks

Risks include over-broad scope, naming drift, underspecified configuration, and treating error handling as an implementation detail. Out of scope: asset import, simulation, rendering, physics, and editor UX.

Leaving those areas out protects the contract layer.

Constraint	Implication
No hidden dependencies	Services are declared in contracts; modules do not locate each other.
No premature runtime design	D001 names the runtime language, not its scheduler.
Stable naming	Module, capability, host, adapter, policy stay consistent.
Explicit failure	Validation and startup failures surface as structured results.
Forward compatibility	Later milestones may add modules but not rewrite foundations.

The foundation should remain reviewable, not sprawling.

13. Verification

Approve D001 only when ownership, naming, dependency flow, and lifecycle language are all consistent and documented. A later milestone must be able to point back to the foundation and explain its dependency.

This is an architecture check, not a code-execution check.

Check	Verification item	Pass
1	Documented ownership for each core concept	Yes/No
2	Contract names are consistent across sections	Yes/No
3	Dependency flow is inward or downward only	Yes/No
4	Error and validation rules are explicit	Yes/No
5	Later milestones have a documented touchpoint	Yes/No
6	Freeze criteria are measurable and actionable	Yes/No

If the team cannot summarize each contract in a sentence or two, the milestone is not ready.

14. Freeze Criteria

Freeze when naming, ownership, dependency flow, integration readiness, and risk closure are all explicit. After that, edits become controlled architecture revisions, not casual changes.

D001 is the point where Echo Engine stops being a loose idea set and becomes a structured system.

Term	Working definition
Host	Outer application that creates and drives the engine.
Composition root	Where concrete services are assembled and injected.
Module	Responsibility boundary with a contract and capabilities.
Capability	Declared feature discoverable by later milestones.
Adapter	Boundary object for host-specific services.
Frozen	Stable enough for downstream dependency.

The next milestone can now build without renegotiating the vocabulary.